React useContext Tutorial

Learning Goals

1. Explain React Context as global-like state sharing within a component tree.
2. Identify prop drilling and why it is a problem.
3. Build the Context solution using `createContext`, Provider, and `useContext`.
4. Extend the baseline example with a practical class challenge.

---

1) React Context

React Context is a way to share state across multiple nested components without passing props through every level.

Context is often used with `useState`.

---

2) The Problem: Prop Drilling

Imagine a tree where the top component and a deeply nested component both need the same data.

If you only use props, middle components must forward values they do not use.

That repeated pass-through is called prop drilling.

Baseline (Prop Drilling) Example

```
const { useState } = React;

function Component1() {
  const [user, setUser] = useState("Linus");

  return (
    <>
      <h1>{`Hello ${user}!`}</h1>
      <Component2 user={user} />
    </>
  );
}
function Component2({ user }) {
  return (
    <>
      <h1>Component 2</h1>
      <Component3 user={user} />
    </>
  );
}
function Component3({ user }) {
  return (
    <>
      <h1>Component 3</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}
```

Observation:

- `Component2` does not really need `user`.
- It only forwards `user` to `Component3`.

---

3) The Solution: Context

The standard solution has three parts:

1. Create context.
2. Provide context value high in the tree.
3. Consume context where needed.

Step A: Create Context

```
const { useState, createContext, useContext } = React;

const UserContext = createContext();
```

Step B: Wrap children in a Provider

```
function Component1() {
  const [user, setUser] = useState("Linus");

  return (
    <UserContext.Provider value={user}>
      <h1>{`Hello ${user}!`}</h1>
      <Component2 />
    </UserContext.Provider>
  );
}
```

Now all descendants inside that provider can access `user`.

Step C: Read context in the deep child

```
function Component3() {
  const user = useContext(UserContext);

  return (
    <>
      <h1>Component 3</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}
```

`Component2` no longer needs to receive or forward `user`.

---

4) Full Example (Context Version)

```
const { useState, createContext, useContext } = React;

const UserContext = createContext();
```

```
function Component1() {
  const [user, setUser] = useState("Linus");

  return (
    <UserContext.Provider value={user}>
      <h1>{`Hello ${user}!`}</h1>
      <Component2 />
    </UserContext.Provider>
  );
}

function Component2() {
  return (
    <>
      <h1>Component 2</h1>
      <Component3 />
    </>
  );
}

function Component3() {
  const user = useContext(UserContext);

  return (
    <>
      <h1>Component 3</h1>
      <h2>{`Hello ${user} again!`}</h2>
    </>
  );
}

const root = ReactDOM.createRoot(document.getElementById("root"));
root.render(<Component1 />);
```

---

5) Classroom Walkthrough (10-12 minutes)

1. Show prop drilling version first.
2. Ask students which component is unnecessary in the data path.
3. Introduce `createContext`.
4. Add Provider at top-level component.
5. Replace prop forwarding with `useContext` in deep child.
6. Compare before/after and discuss readability.

---

6) Useful Challenge

Challenge: Share Two Values with Context

Expand the example to share:

1. `user`
2. `course`

Requirements:

- Top component can change both values.
- Deep child displays both values.
- Middle component receives no props.

---

7) Step-by-Step Challenge Solution

Step 1: Hold both values in state

```
const [user, setUser] = useState("Linus");
const [course, setCourse] = useState("React Fundamentals");
```

Step 2: Provide an object instead of a single string

```
<UserContext.Provider value={{ user, course, setUser, setCourse }}>
  <Component2 />
</UserContext.Provider>
```

Step 3: Consume object values in deep child

```
const { user, course } = useContext(UserContext);
```

Step 4: Add update controls in top component

```
<button onClick={() => setUser("Ada")}>Set Ada</button>
<button onClick={() => setCourse("Frontend Architecture")}>Set Course</button>
```

Step 5: Verify behavior

1. Update user and course from top.
2. Confirm deep child updates instantly.
3. Confirm middle component still has no prop forwarding.

---

8) Common Student Questions

1. Can context share data with siblings? Yes. If both siblings are inside the same provider, each can read the same context.
2. Does context replace props everywhere? No. Use props for local parent-child communication; use context when many levels/components need the same data.
3. Should context hold everything? No. Keep context focused on shared concerns.

---

9) Instructor Notes

1. Teach props first, then context as a targeted solution.
2. Emphasize that context reduces prop drilling, not component rendering by itself.
3. Link this lesson to your `useMemo` lesson for provider value optimization in larger apps.